

TripCraft

관광지 탐색부터 드래그앤드롭 일정 확정, 이동시간 자동 계산, 커뮤니티 공유까지

팀원: 전진 · 송정기

2026.06

목차

1. 기획 배경 · 목표
2. 추진 계획 — 팀 전체 / 개인별 일정
3. 시장 분석 — 경쟁 비교 · 차별화
4. 개발 결과 — 핵심 기술 · 구현
5. 개발 환경 & 시스템 구조도
6. 화면 흐름도 및 시연
7. 적용 패턴 및 핵심 알고리즘
8. 기대 효과
9. 개발 후기

1. 기획 배경 · 목표

문제 인식

- 여행 계획 시 관광지 검색 → 지도 → 길찾기 앱을 여러 서비스를 오가며 반복
- 후보 장소를 모아두고 하루 단위로 동선을 짜는 도구 부재
- 장소 간 현실적 이동 시간을 일정에 반영하기 어려움

목표 — "한 화면에서 끝나는 여행 일정 도구"

- 관광지 탐색 → 후보군 → 드래그앤드롭 일정 확정을 한 흐름으로
- 구간별 이동 시간·수단 자동 계산 및 경로 지도 시각화
- 완성한 일정을 커뮤니티로 공유, 일행과 실시간 협업

추가 기능 중심: 이동시간 자동계산 · 실시간 협업 · AI 챗봇 · 방문 지도

2. 추진 계획 — 팀 전체 일정

단계	기간	내용
기획	W1	요구사항·WBS·환경 세팅
설계	W2	ERD·API·와이어프레임·컨벤션
개발	W3~5	회원 → 관광지·즐거찾기 → 일정·이동시간
고도화	W6~	커뮤니티·협업·AI 챗봇·마이페이지
QA·정리	막바지	통합테스트·보안·산출물

✅ M1 설계 → M2 회원+관광지 → M3 일정 → M4 MVP+고도화 → 🔄 M5 제출

2. 추진 계획 — 개인별 일정

주차	전진	송정기
W1~2	요구사항·유스케이스·API 명세	ERD·스키마·컨벤션·환경
W3	관광지 TourAPI 수집·조회	회원·JWT·Spring Security
W4	탐색 UI·네이버맵	즐거찾기·후보군 자동연동
W5	이동시간(ODsay/T Map)·경로 시각화	일정 드래그앤드롭 타임라인
W6~	AI 챗봇·커뮤니티	실시간 협업·공유링크·마이페이지

핵심 화면(일정 편집)·협업은 공동 작업, 세부 분담은 Git/MR 이력 기준.

3. 시장 분석 — 경쟁 비교

항목	네이버 여행	구글 지도	트리플	TripCraft
관광지 탐색	○	○	○	○
드래그앤드롭 일정	△	×	△	◎
구간 이동시간 자동	×	○	△	◎
이동수단 구간별 선택	×	△	×	◎
실시간 공동 편집	×	×	×	◎
AI 관광 챗봇	×	×	△	◎
일정 커뮤니티 공유	△	×	○	○

◎ 강점 · ○ 지원 · △ 부분 · × 미지원

3. 시장 분석 — 차별화 전략

- 통합 동선 설계: 탐색-후보군-타임라인을 한 화면(작업실)에서 끊김 없이
- 현실적 이동시간: ODsay(대중교통) + T Map(자동차·도보·택시요금)을 구간별 모드 선택으로
- 협업 우선: 공유 링크 + 실시간 동시 편집(낙관적 락) — 경쟁 서비스 미제공 영역
- AI 도우미: 관광지 컨텍스트 Q&A + 반경 3km 주변 추천
- 기록의 자산화: 여행 이야기 → 방문 지도로 시각화

4. 개발 결과 — 핵심 기술 (1/2)

관광지 도메인

- 한국관광공사 TourAPI 4.0 전국 데이터 일괄 수집 → 자체 DB 서비스
- 지역.카테고리.키워드 조회, 상세(소개.이미지.이용안내), 네이버맵 마커 연동

일정 도메인

- Trip → Candidate → Block 모델, vue-draggable 드래그앤드롭 타임라인
- 후보군 도시(시군구) 자동 분류, 즐겨찾기 자동 연동, 중복 방문 허용

4. 개발 결과 — 핵심 기술 (2/2)

이동시간 · 경로

- ODsay·T Map 연동, 구간별 모드(대중교통/자동차/도보), 택시요금, 좌표 기반 모드별 캐시
- 대중교통 노선 폴리라인 + 환승/도보 구간 시각화

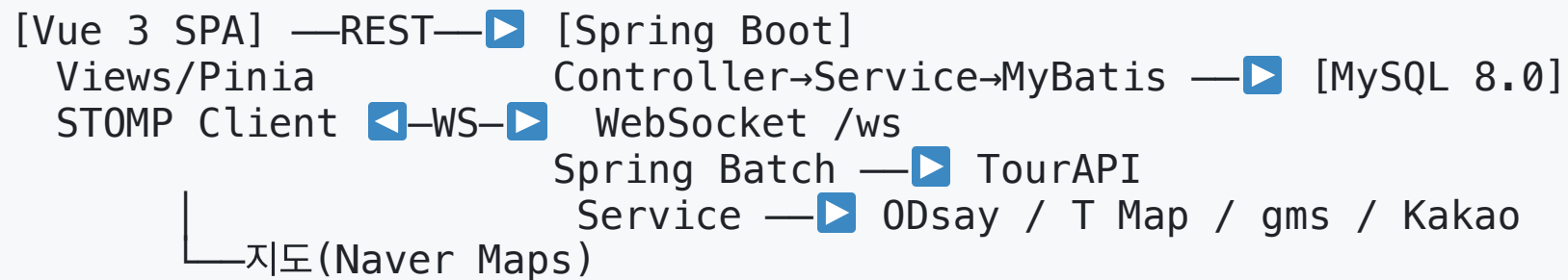
협업 · 커뮤니티 · AI

- WebSocket(STOMP) 실시간 협업 + 공유 링크(VIEW/EDIT)
- 게시판·공지·댓글·좋아요·북마크, 방문 지도
- Spring AI + gms(gpt-4.1) 관광 챗봇, 주변 추천

5. 개발 환경 & 기술 스택

Layer	기술
Backend	Java 21 · Spring Boot 3.5 · Spring Security/Batch/AI · MyBatis · MySQL 8 · Gradle
Frontend	Vue 3 + Vite · Pinia · Vue Router · vue-draggable · Tiptap
인증	JWT(30분/7일) · BCrypt · Kakao OAuth
실시간	WebSocket + STOMP + SockJS
외부 API	TourAPI · ODsay · T Map · Naver Maps · Kakao · gms(gpt-4.1)

5. 전체 시스템 구조도



- 공통 응답 `ApiResponse{success,data,message,errorCode}`
- JWT 필터 인증, 서버 측 소유권/역할 검증
- 외부 API 키는 전부 환경변수 주입

상세: `05_reference/기타참고문서.md`, ER/클래스: `02_diagrams/`

6. 화면 흐름도 및 시연

흐름: 랜딩 → (로그인/카카오) → 여행 작업실 ↔ 여행 이야기 ↔ 마이페이지

- 작업실(/plan): 탐색 모드(검색.후보군) ↔ 정리 모드(드래그 타임라인.이동시간.지도)
- 여행 이야기(/community): 목록 → 상세(좋아요.북마크.댓글) / 글쓰기(Tiptap)
- 마이페이지(/mypage): 여행.내정보.내장소.방문지도.내가쓴글.북마크.좋아요

시연 순서: 회원가입 → 관광지 탐색.후보군 → 드래그 일정 → 이동시간.지도 → 공유.협업 → AI 챗봇 → 방문 지도

(시연 캡처/영상 삽입: [채우세요])

7. 적용 패턴 및 핵심 알고리즘 (1/2)

이동시간 캐싱

- 키 = (좌표 route_key, 출발시각대, 이동수단 모드) → 모드별 독립 캐시
- 캐시 정밀도 1~5단계(`system_config`), 러시아워 분기 → 무료 API 쿼터 보호
- 노선 형상은 `lane_polyline` 영구 캐시

드래그앤드롭 타임라인

- 후보군 → Day 그리드 드롭으로 블록 생성, 30분 스냅, 사이드바 삭제존
- 날짜 범위 이탈은 DB TRIGGER로 방어

7. 적용 패턴 및 핵심 알고리즘 (2/2)

실시간 협업 — 낙관적 락

- `trip_block.version` 으로 동시 편집 충돌 감지 → 후행 저장 거부·재동기화
- transit 재계산은 version 미변경(불필요한 충돌 방지)

AI 주변 추천

- `ST_Distance_Sphere` 로 반경 3km 거리순 조회 → 프롭프트 주입
- 응답 언급 장소만 버튼화 → 지도 핀·상세 이동, 뒤로가기 복원

데이터 보존

- 작성자/일정 삭제 SET NULL 보존, 글 소프트 딜리트 + 북마크 보존, 후보군 RESTRICT

8. 기대 효과

- 여행 준비 시간 단축: 탐색-동선-이동시간을 한 화면에서 → 앱 전환 비용 제거
- 현실적인 일정: 구간별 실제 이동시간·수단으로無理 없는 동선
- 함께 만드는 여행: 실시간 협업·공유로 일행과 합의된 일정
- 경험의 축적: 여행 이야기·방문 지도로 기록을 자산화
- 확장성: 도메인 중심 설계 + 캐시 전략으로 트래픽·기능 확장 용이

9. 개발 후기

팀 사진

[팀 사진 삽입: 채우세요]

개인별 회고

- 전진: *[회고 작성: 채우세요]*
- 송정기: *[회고 작성: 채우세요]*

기술적 배움(예시 — 보완)

- 외부 API 쿼터 한계를 캐시 설계로 극복
- 실시간 동시 편집의 충돌 처리(낙관적 락) 경험
- 도메인 중심 패키지·삭제 전략 설계의 중요성

감사합니다

TripCraft — 전진 · 송정기